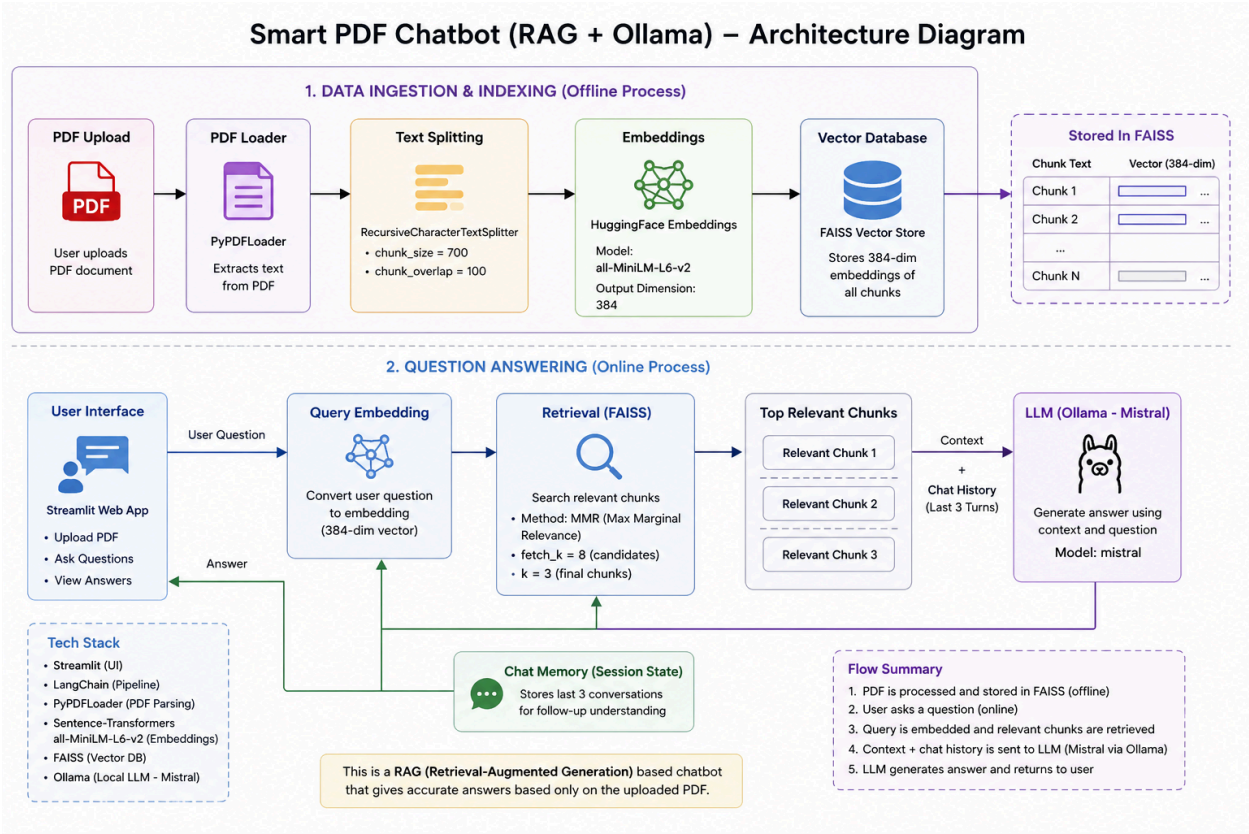
Smart PDF Chatbot (RAG + Ollama) project.

1. **PDF is loaded using PyPDFLoader**
2. **Text is split into chunks (700 size, 100 overlap)**
3. **Each chunk is converted into 384-dim embeddings**
4. **Stored in FAISS vector database**
5. **User query is converted into embedding**
6. **FAISS retrieves top relevant chunks using MMR**
7. **Retrieved context is passed to Mistral (Ollama)**
8. **Model generates final answer**

Architecture



1. Basic Questions

What is your project about?

My project is a Smart PDF Chatbot that allows users to upload any PDF and ask questions about its content. The system reads the document, processes it, and gives answers using a retrieval-augmented generation approach.

What problem does your project solve?

It solves the problem of manually searching large documents. Instead of reading the full PDF, users can directly ask questions and get relevant answers instantly.

Why did you choose RAG for this project?

I chose RAG because it combines document retrieval and language generation. This helps the model give accurate answers based on the uploaded document

instead of generating random responses.

What tools and frameworks did you use?

I used Streamlit for UI, LangChain for building the pipeline, FAISS for vector storage, Hugging Face for embeddings, and Ollama for running a local language model.

What is Streamlit used for?

Streamlit is used to create the web interface where users can upload PDFs, type questions, and view answers in a simple and interactive way.

What is FAISS?

FAISS is a vector database used to store embeddings of text chunks and quickly find similar content based on user queries.

What is Ollama?

Ollama is a tool that allows running large language models locally on the system without needing external APIs.

2. Conceptual Questions

How do you extract text from PDF?

I use PyPDFLoader from LangChain. It reads the PDF file and converts each page into text format for further processing.

What is chunking and why is it required?

Chunking means splitting large text into smaller parts. It is required because language models and embeddings work better with smaller, meaningful pieces of text.

What happens if you don’t split documents into chunks?

If we don’t split the document, the model may miss important information, and retrieval becomes less accurate due to large and unstructured input.

What are embeddings in NLP?

Embeddings are numerical representations of text. They convert words or sentences into vectors so that machines can understand semantic meaning.

Why do we use all-MiniLM-L6-v2 model?

This model is lightweight, fast, and produces good quality embeddings, making it suitable for real-time applications like chatbots.

What is vector similarity?

Vector similarity measures how close two text embeddings are. It helps find the most relevant document chunks for a query.

What is RAG?

RAG stands for Retrieval-Augmented Generation. It first retrieves relevant information from a database and then uses a language model to generate an answer based on that information.

How does RAG improve LLM responses?

RAG improves accuracy by grounding responses in real document data instead of relying only on the model’s trained knowledge.

What is retrieval and generation in your pipeline?

Retrieval means fetching relevant text chunks from FAISS, and generation means using the LLM to create a final answer based on that context.

3. Technical Deep Questions

How does FAISS store vectors?

FAISS stores embeddings in a high-dimensional space and performs fast similarity search using indexing techniques.

Why did you choose FAISS over other vector databases?

I chose FAISS because it is lightweight, fast, works offline, and is easy to integrate with Python.

What is similarity search vs MMR?

Similarity search returns the most similar documents. MMR (Max Marginal Relevance) improves diversity by avoiding repeated or similar chunks.

What is Ollama?

Ollama is a local LLM runtime that allows running models like Mistral or Llama on your system without cloud APIs.

Why did you use a local LLM instead of OpenAI API?

I used a local LLM to avoid API costs, improve privacy, and make the system fully offline.

How does Mistral generate answers from context?

Mistral takes the retrieved context and the question as input, then generates an answer based on patterns learned during training.

Explain end-to-end flow of your system

First, the PDF is uploaded. Then it is split into chunks. These chunks are converted into embeddings and stored in FAISS. When a user asks a question, it is also converted into embeddings. FAISS retrieves similar chunks, and these chunks are sent to the LLM, which generates the final answer.

What happens when a user asks a question?

The question is converted into a vector, relevant document chunks are retrieved, and the LLM generates an answer using that context.

How is previous chat history used?

Previous questions and answers are added to the prompt so the system can understand follow-up questions like “explain it” or “summarize it”.

4. Advanced Questions

How will your system handle large PDFs?

It can handle large PDFs by increasing chunking efficiency, using better indexing, and optimizing retrieval parameters.

How can you improve retrieval quality?

I can improve it by tuning chunk size, using better embedding models, or combining semantic and keyword search.

What is the limitation of your system?

The system depends on retrieval quality. If relevant chunks are not retrieved, the answer may be incomplete.

How would you scale this project?

I would move the vector database to a cloud service like Pinecone and deploy the backend using FastAPI with load balancing.

How would you deploy it in production?

I would containerize the app using Docker and deploy it on cloud platforms like AWS or Streamlit Cloud.

5. Debugging Questions

Why did you choose Streamlit?

Because it allows fast development of web apps with minimal frontend code.

What issues did you face?

I faced dependency issues with sentence-transformers and torch, and I resolved them by installing proper versions.

What happens if embeddings fail?

If embeddings fail, the system cannot perform similarity search, and retrieval will not work.

How do you handle irrelevant answers?

I restrict the LLM using strict system prompts and ensure answers are based only on retrieved context.

6. Scenario-Based Questions

If user asks “explain it” without context?

The system uses previous chat history to understand what “it” refers to and retrieves relevant context accordingly.

If PDF does not contain answer?

The system responds that the answer is not found in the document.

How does chatbot maintain memory?

It stores last few question-answer pairs in session state and includes them in the prompt.

Vector Size (Embedding Dimension)

I used:

Model: `sentence-transformers/all-MiniLM-L6-v2`

Vector size:

Each text chunk is converted into a:

384-dimensional vector

- Every sentence is converted into a list of 384 numbers
- These numbers represent the meaning of the text
- Similar meanings → vectors are closer in space

Embedding Model Details

I used:

```
HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2"
)
```

Key properties:

- Lightweight model
- Fast inference
- Good semantic understanding
- Output dimension = 384
- Suitable for CPU-based systems

Chunking Parameters

I used:

```
RecursiveCharacterTextSplitter(
    chunk_size=700,
    chunk_overlap=100
)
```

chunk_size = 700

- Each text chunk contains ~700 characters
- Keeps context small and meaningful

chunk_overlap = 100

- 100 characters are repeated between chunks
- Helps preserve context between splits

Vector Database

I used:

FAISS (Facebook AI Similarity Search)

What it stores:

- 384-dimension embeddings

- Each chunk of PDF text as a vector

Why FAISS:

- Fast similarity search
- Works offline
- Easy integration with LangChain

Retrieval Parameters

I used:

```
db.max_marginal_relevance_search(query,k=3,fetch_k=8)
```

Parameters used :

k = 3

- Final top 3 most relevant chunks are selected
- These are sent to LLM

fetch_k = 8

- First retrieves 8 candidate chunks
- Then applies MMR filtering

Retrieval Method

I used:

Max Marginal Relevance (MMR)

Why MMR:

- Avoids repeated information

- Ensures diversity in retrieved chunks
- Improves answer quality

LLM Model

I used:

```
ollama model = "mistral"
```

What it does:

- Takes retrieved context
- Generates final answer
- Runs locally on your system

Prompt Engineering Parameters

System prompt rules:

- Use ONLY given context
- If answer not found → say “Not found in document”
- Keep answers simple and structured
- Handle follow-up questions using chat history

Chat Memory

I used:

```
st.session_state.chat_history[-3:]
```

Meaning:

- Stores last 3 interactions
- Helps understand follow-up questions like:
  - “explain it”
  - “summarize it”